Write a code using Terraform that deploys and achieves the following:-

1. **Create an EC2 Instance on AWS Cloud using the below-mentioned ubuntu AMI and instance_type. Or,**
   (You can use of your choice as well) ami = "ami-0461b1436c9b3c1a3"
   instance_type = "t2.micro"

Step 1: Create the project directory

Created a project directory named terraform-cricbuzz to maintain all Terraform configuration and script files.

Step 2: Create Terraform configuration files

Created the Terraform configuration file main.tf to define the AWS provider and infrastructure resources.

The main resources used included:

- aws_instance.cricbuzz
- aws_security_group for application access
- IAM role and instance profile for SSM access
- IAM policy attachment for Systems Manager permissions

Step 3: Configure AWS Systems Manager access

Created the IAM role:

Cricbuzz-EC2-SSM-Role

Created the instance profile:

Cricbuzz-EC2-SSM-Profile

Attached the required SSM permissions and associated the instance profile with the EC2 instance.

This allowed us to execute commands on the EC2 instance using AWS Systems Manager without using SSH.

Step 4: Configure the EC2 instance

Created the Ubuntu EC2 instance using:

```
resource "aws_instance" "cricbuzz" {

  # Ubuntu AMI and EC2 configuration
```

```
  # Instance type: t3.micro
  # IAM instance profile: Cricbuzz-EC2-SSM-Profile
}
```

The actual EC2 resource was configured in main.tf along with its AMI, security group, IAM instance profile and other required settings.

Step 5: Execute Terraform commands

terraform init

terraform fmt

terraform validate

terraform plan

terraform apply

Step 6: Verify the EC2 instance

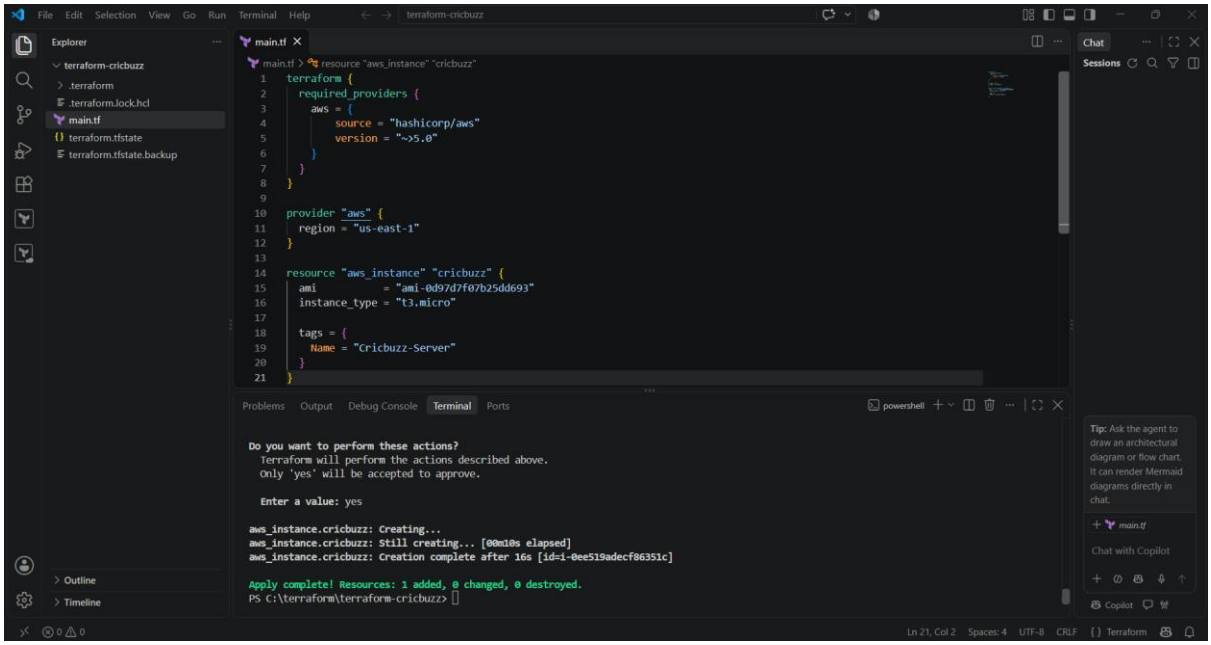
Verified that the EC2 instance was created successfully in AWS.

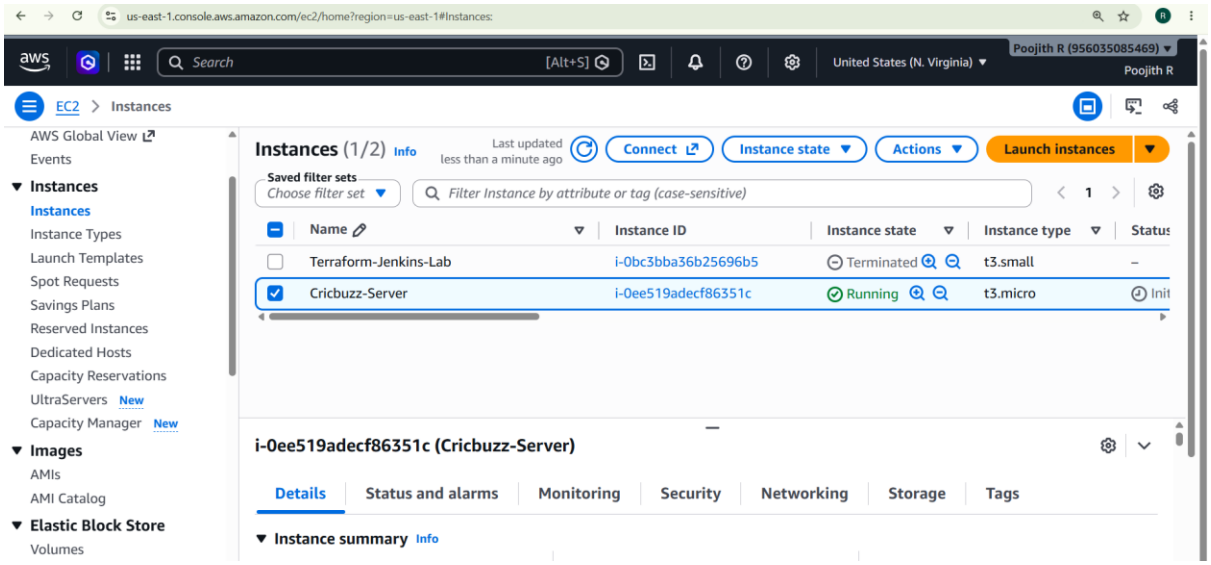
Used AWS Systems Manager to connect to the instance and execute commands.

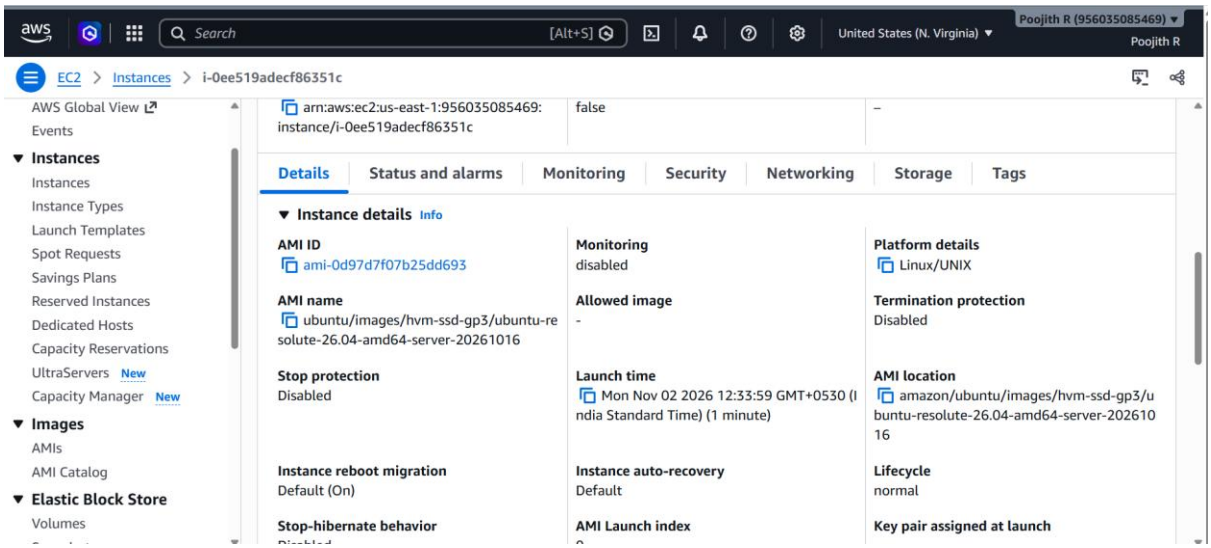
EC2 Instance ID: i-02e7538bb06c03d92

Public IP: 100.26.48.41

Result: Ubuntu EC2 instance created successfully.







2. **Install a web server on the same EC2 Instance. The web server should listen on port 8080.**

Step 1: Create the Nginx installation script

Created a file named:

install-nginx.sh

Used the following script:

#!/bin/bash

```
set -eux

apt-get update -y
apt-get install -y nginx

sed -i 's/listen 80 default_server;/listen 8080 default_server;/g' /etc/nginx/sites-available/default
sed -i 's/listen \[::\]:80 default_server;/listen [::]:8080 default_server;/g' /etc/nginx/sites-available/default

nginx -t

systemctl enable nginx
systemctl restart nginx
```

Explanation:

- Updated the package repository.
- Installed Nginx.
- Changed the default listening port from 80 to 8080.
- Validated the Nginx configuration.
- Enabled and restarted the Nginx service.

Step 2: Configure the Terraform local variable

In main.tf, used a local variable to read and normalize the script:

```
locals {
  nginx_script = trimspace(
    replace(
      replace(file("${path.module}/install-nginx.sh"), "\r\n", "\n"),
      "\uFEFF",
      ""
    )
```

)

}

This removed Windows line endings and any UTF-8 BOM from the script.

Step 3: Execute the script using AWS Systems Manager

Created the Terraform resource:

aws_ssm_association.install_nginx

Used the AWS document:

AWS-RunShellScript

The script was encoded and executed using:

printf '%s' '${base64encode(local.nginx_script)}' | base64 -d | bash

The association targeted:

targets {

key = "InstanceIds"

values = [aws_instance.cricbuzz.id]

}

The association was configured with a 600-second execution timeout and depended on the SSM IAM policy attachment.

Step 4: Verify Nginx

Executed the following commands through SSM:

sudo nginx -t

sudo systemctl status nginx

sudo ss -tlnp

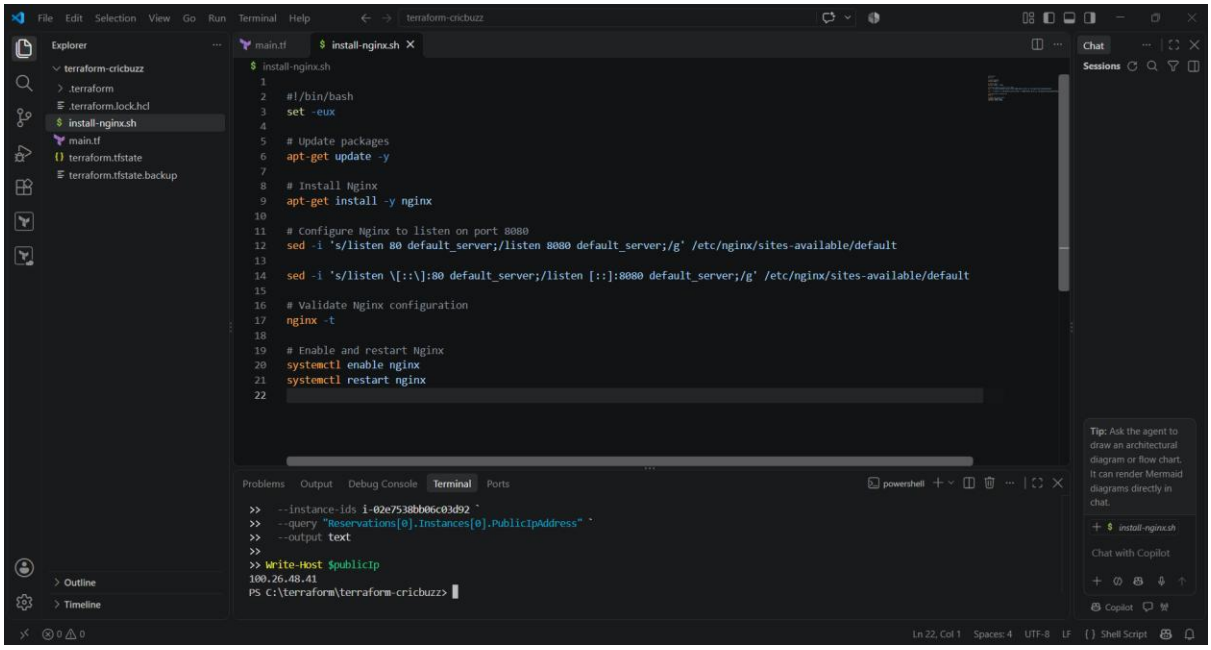
Step 5: Browser verification

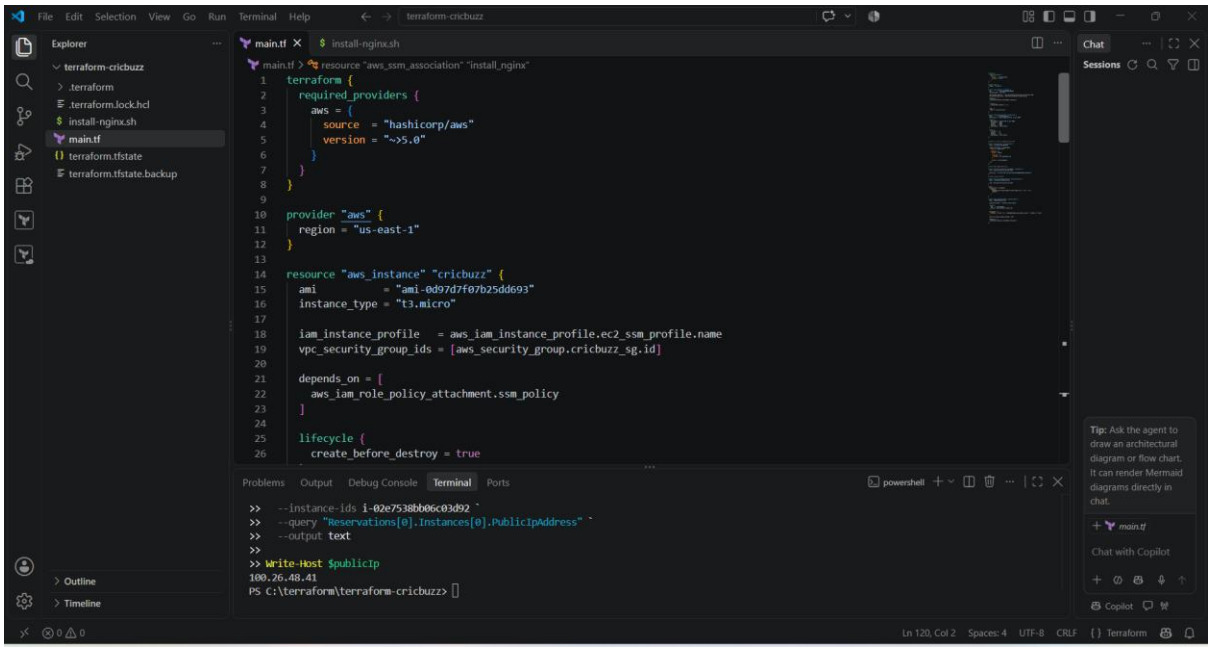
Opened:

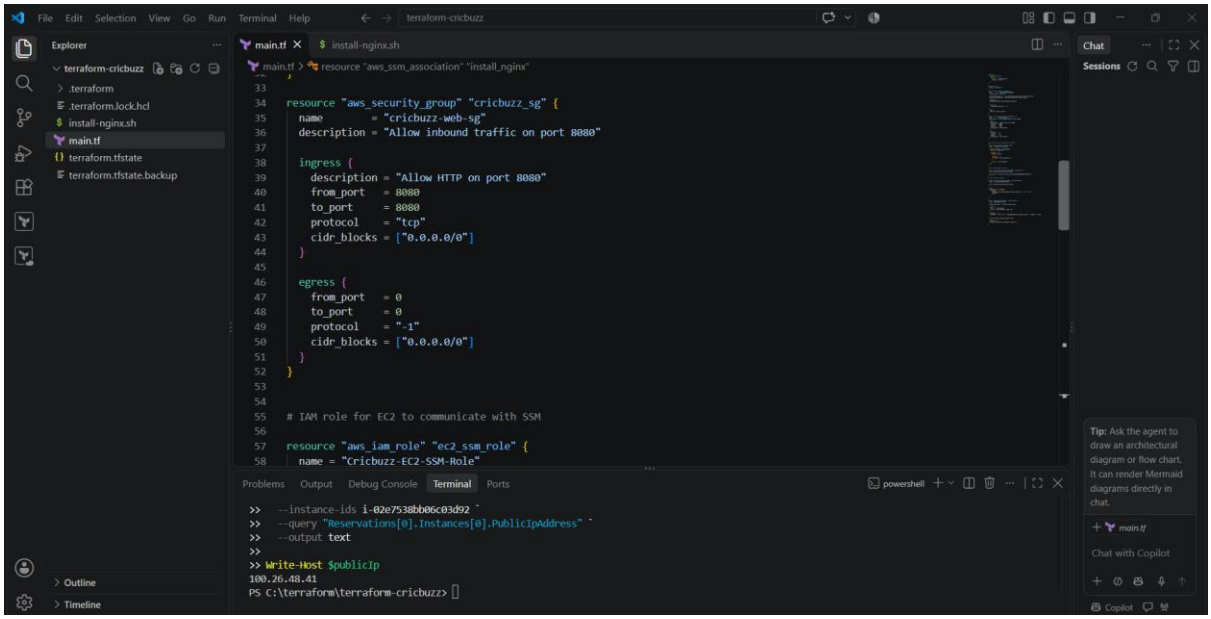
http://100.26.48.41:8080

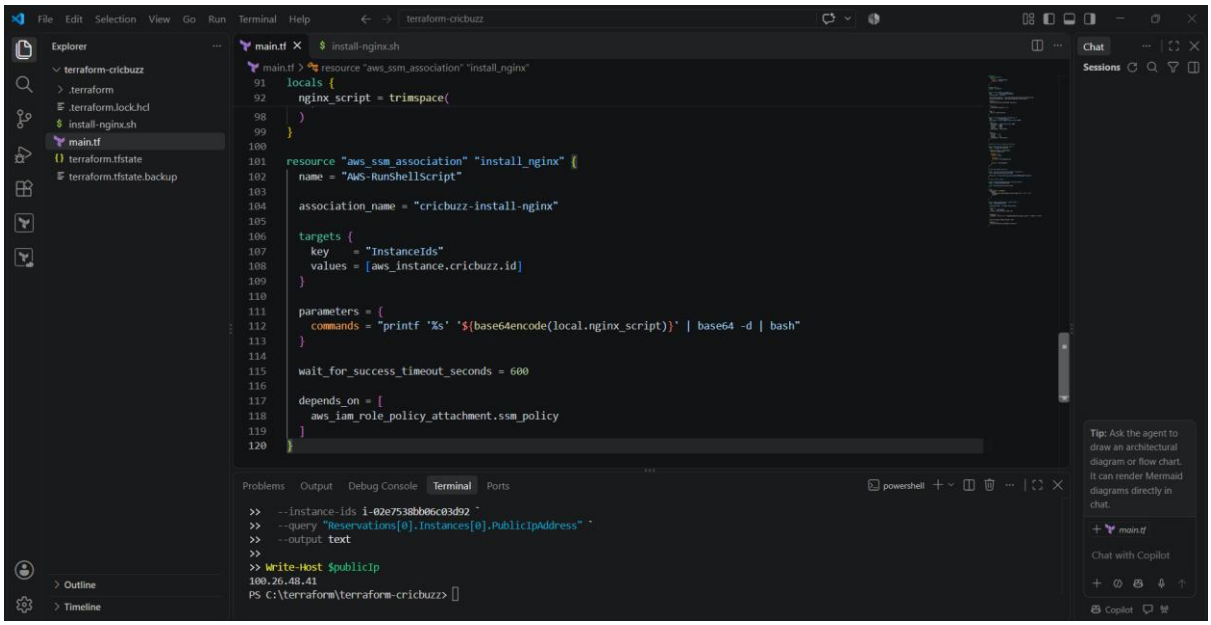
The Nginx welcome page was displayed successfully.

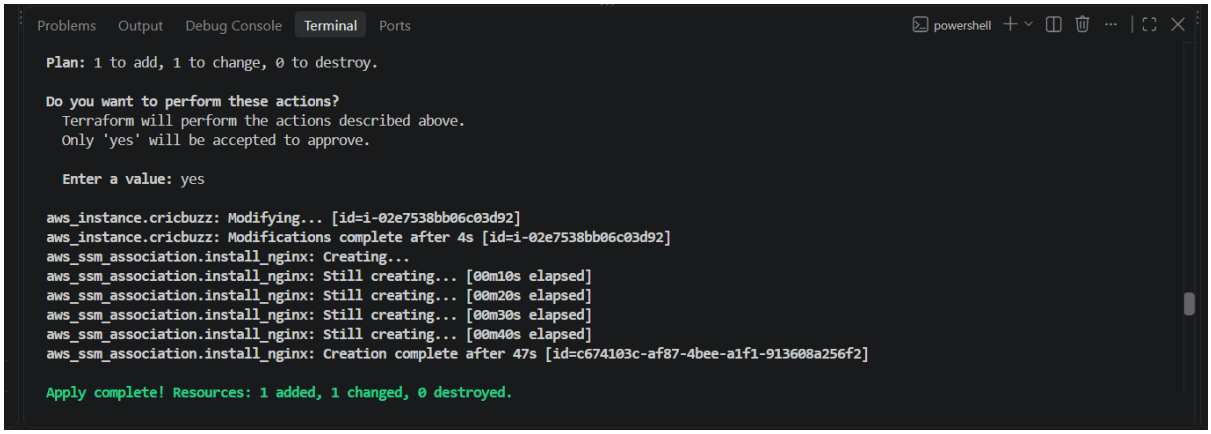
Result: Nginx installed successfully and configured to listen on port 8080.


File Edit Selection View Go Run Terminal Help
terraform-cricbuzz
Explorer
terraform-cricbuzz
.terraform
.terraform.lock.hcl
install-nginx.sh
main.tf
terraform.tfstate
terraform.tfstate.backup
main.tf
install-nginx.sh
install-nginx.sh
#!/bin/bash
set -eux
Update packages
apt-get update -y
Install Nginx
apt-get install -y nginx
Configure Nginx to listen on port 8080
sed -i 's/listen 80 default_server;/listen 8080 default_server;/g' /etc/nginx/sites-available/default
sed -i 's/listen \[::\]:80 default_server;/listen [::]:8080 default_server;/g' /etc/nginx/sites-available/default
Validate Nginx configuration
nginx -t
Enable and restart Nginx
systemctl enable nginx
systemctl restart nginx
Problems Output Debug Console Terminal Ports
powershell
>> --instance-ids i-02e7538bb06c03d92 `
>> --query "Reservations[0].Instances[0].PublicIpAddress" `
>> --output text
>>
>> Write-Host $publicIp
100.26.48.41
PS C:\terraform\terraform-cricbuzz>
Outline
Timeline
Chat
Sessions
Tip: Ask the agent to draw an architectural diagram or flow chart. It can render Mermaid diagrams directly in chat.
install-nginx.sh
Chat with Copilot
Copilot
Ln 22, Col 1 Spaces: 4 UTF-8 LF Shell Script


File Edit Selection View Go Run Terminal Help
terraform-cricbuzz
Explorer
terraform-cricbuzz
.terraform
.terraform.lock.hcl
install-nginx.sh
main.tf
terraform.tfstate
terraform.tfstate.backup
main.tf
install-nginx.sh
main.tf > resource "aws_ssm_association" "install_nginx"
terraform {
required_providers {
aws = {
source = "hashicorp/aws"
version = "~>5.0"
}
}
}
provider "aws" {
region = "us-east-1"
}
resource "aws_instance" "cricbuzz" {
ami = "ami-0d97d7f07b25dd693"
instance_type = "t3.micro"
iam_instance_profile = aws_iam_instance_profile.ec2_ssm_profile.name
vpc_security_group_ids = [aws_security_group.cricbuzz_sg.id]
depends_on = [
aws_iam_role_policy_attachment.ssm_policy
]
lifecycle {
create_before_destroy = true
Problems Output Debug Console Terminal Ports
powershell
>> --instance-ids i-02e7538bb06c03d92 `
>> --query "Reservations[0].Instances[0].PublicIpAddress" `
>> --output text
>>
>> Write-Host $publicIp
100.26.48.41
PS C:\terraform\terraform-cricbuzz>
Outline
Timeline
Chat
Sessions
Tip: Ask the agent to draw an architectural diagram or flow chart. It can render Mermaid diagrams directly in chat.
main.tf
Chat with Copilot
Copilot
Ln 120, Col 2 Spaces: 4 UTF-8 CRLF Terraform


File Edit Selection View Go Run Terminal Help
terraform-cricbuzz
Explorer
terraform-cricbuzz
.terraform
.terraform.lock.hcl
install-nginx.sh
main.tf
terraform.tfstate
terraform.tfstate.backup
main.tf
install-nginx.sh
main.tf > resource "aws_ssm_association" "install_nginx"
resource "aws_security_group" "cricbuzz_sg" {
name = "cricbuzz-web-sg"
description = "Allow inbound traffic on port 8080"
ingress {
description = "Allow HTTP on port 8080"
from_port = 8080
to_port = 8080
protocol = "tcp"
cidr_blocks = ["0.0.0.0/0"]
}
egress {
from_port = 0
to_port = 0
protocol = "-1"
cidr_blocks = ["0.0.0.0/0"]
}
}
IAM role for EC2 to communicate with SSM
resource "aws_iam_role" "ec2_ssm_role" {
name = "Cricbuzz-EC2-SSM-Role"
Problems Output Debug Console Terminal Ports
powershell
>> --instance-ids i-02e7538bb06c03d92 `
>> --query "Reservations[0].Instances[0].PublicIpAddress" `
>> --output text
>>
>> Write-Host $publicIp
100.26.48.41
PS C:\terraform\terraform-cricbuzz>
Chat
Sessions
Tip: Ask the agent to draw an architectural diagram or flow chart. It can render Mermaid diagrams directly in chat.
main.tf
Chat with Copilot
Copilot
Outline
Timeline


File Edit Selection View Go Run Terminal Help
terraform-cricbuzz
Explorer
terraform-cricbuzz
.terraform
.terraform.lock.hcl
install-nginx.sh
main.tf
terraform.tfstate
terraform.tfstate.backup
main.tf
install-nginx.sh
main.tf > resource "aws_ssm_association" "install_nginx"
locals {
nginx_script = trimspace(
)
}
resource "aws_ssm_association" "install_nginx" {
name = "AWS-RunShellScript"
association_name = "cricbuzz-install-nginx"
targets {
key = "InstanceIds"
values = [aws_instance.cricbuzz.id]
}
parameters = {
commands = "printf '%s' '${base64encode(local.nginx_script)}' | base64 -d | bash"
}
wait_for_success_timeout_seconds = 600
depends_on = [
aws_iam_role_policy_attachment.ssm_policy
]
}
Problems Output Debug Console Terminal Ports
powershell
>> --instance-ids i-02e7538bb06c03d92 `
>> --query "Reservations[0].Instances[0].PublicIpAddress" `
>> --output text
>>
>> Write-Host $publicIp
100.26.48.41
PS C:\terraform\terraform-cricbuzz>
Chat
Sessions
Tip: Ask the agent to draw an architectural diagram or flow chart. It can render Mermaid diagrams directly in chat.
main.tf
Chat with Copilot
Copilot
Outline
Timeline


Problems Output Debug Console Terminal Ports
powershell
Plan: 1 to add, 1 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_instance.cricbuzz: Modifying... [id=i-02e7538bb06c03d92]
aws_instance.cricbuzz: Modifications complete after 4s [id=i-02e7538bb06c03d92]
aws_ssm_association.install_nginx: Creating...
aws_ssm_association.install_nginx: Still creating... [00m10s elapsed]
aws_ssm_association.install_nginx: Still creating... [00m20s elapsed]
aws_ssm_association.install_nginx: Still creating... [00m30s elapsed]
aws_ssm_association.install_nginx: Still creating... [00m40s elapsed]
aws_ssm_association.install_nginx: Creation complete after 47s [id=c674103c-af87-4bee-a1f1-913608a256f2]
Apply complete! Resources: 1 added, 1 changed, 0 destroyed.

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

aws_ssm_association.install_nginx: Still creating... [00m40s elapsed]
aws_ssm_association.install_nginx: Creation complete after 47s [id=c674103c-af87-4bee-a1f1-913608a256f2]

Apply complete! Resources: 1 added, 1 changed, 0 destroyed.
PS C:\terraform\terraform-cricbuzz> $commandId = aws ssm send-command `
>>   --instance-ids i-02e7538bb06c03d92 `
>>   --document-name "AWS-RunShellScript" `
>>   --parameters 'commands=["systemctl is-active nginx","sudo ss -lntp | grep 8080","sudo nginx -t"]' `
>>   --query "Command.CommandId" `
>>   --output text
>>
>> Start-Sleep -Seconds 5
>>
>> aws ssm get-command-invocation `
>>   --command-id $commandId `
>>   --instance-id i-02e7538bb06c03d92 `
>>   --output json
{
```

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

PS C:\terraform\terraform-cricbuzz>
PS C:\terraform\terraform-cricbuzz>
PS C:\terraform\terraform-cricbuzz>
PS C:\terraform\terraform-cricbuzz> $publicIp = aws ec2 describe-instances `
>>   --instance-ids i-02e7538bb06c03d92 `
>>   --query "Reservations[0].Instances[0].PublicIpAddress" `
>>   --output text
>>
>> Write-Host $publicIp
100.26.48.41
PS C:\terraform\terraform-cricbuzz>
```

Not secure 100.26.48.41:8080

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

3. **Create a Docker-compose file for the project: https://github.com/betawins/cricbuzz-docker-hello-world.git and point it to port 8000.**

Step 1: Create the Docker installation script

Created a file named:

install-docker-compose.sh

The script installed Docker, Docker Compose and Git, started Docker, cloned the application repository and deployed the container.

```
#!/bin/bash

set -eux


apt-get update -y


apt-get install -y docker.io docker-compose-v2 git


systemctl enable docker

systemctl start docker


docker --version

docker compose version


mkdir -p /opt/cricbuzz


if [ ! -d /opt/cricbuzz/cricbuzz-docker-hello-world ]; then

   git clone https://github.com/betawins/cricbuzz-docker-hello-world.git /opt/cricbuzz/cricbuzz-docker-hello-world

fi


cd /opt/cricbuzz/cricbuzz-docker-hello-world


cat > docker-compose.yml <<'EOF'

services:

  cricbuzz:
```

```
  image: ghcr.io/infrastructure-as-code/hello-world:latest
  ports:
   - "8000:8080"
  restart: unless-stopped
EOF

docker compose up -d
```

Explanation:

- Installed Docker and Docker Compose.
- Installed Git.
- Enabled and started Docker.
- Cloned the Cricbuzz application repository.
- Created the docker-compose.yml file.
- Used the application image ghcr.io/infrastructure-as-code/hello-world:latest.
- Mapped host port 8000 to container port 8080.
- Started the application in detached mode.

Step 2: Configure the Terraform local variable

Added the following local variable in main.tf:

```
locals {
 docker_compose_script = trimspace(
  replace(
   replace(file("${path.module}/install-docker-compose.sh"), "\r\n", "\n"),
   "\uFEFF",
   ""
  )
 )
}
```

This normalized the script before sending it to the Linux instance.

Step 3: Execute the script using SSM

Created the Terraform resource:

aws_ssm_association.install_docker_compose

Used AWS-RunShellScript to execute the script on the EC2 instance.

The execution command was:

printf '%s' '${base64encode(local.docker_compose_script)}' | base64 -d | bash

The association targeted the EC2 instance and had a 600-second timeout.

It was configured to depend on:

aws_ssm_association.install_nginx

Step 4: Verify Docker deployment

Used the following commands on the EC2 instance:

docker ps

docker images

docker compose ps

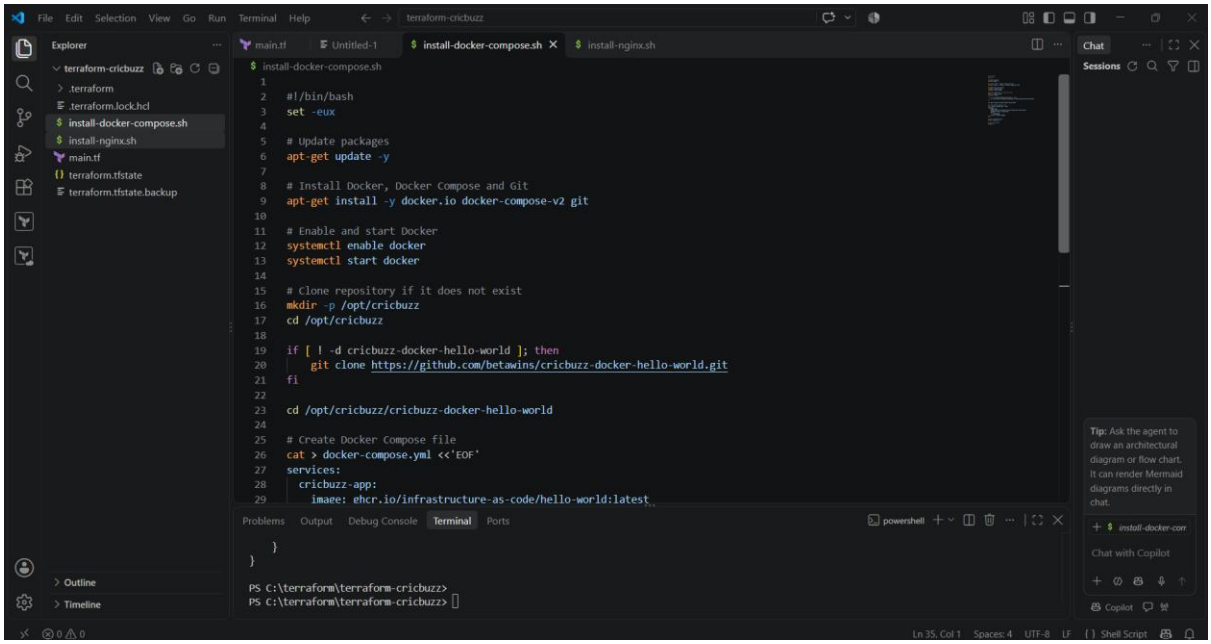
Step 5: Browser verification

Opened:

http://100.26.48.41:8000

The Hello World application was displayed successfully.

Result: Docker Compose application deployed successfully and accessible on port 8000.



```
#!/bin/bash
set -eux

# Update packages
apt-get update -y

# Install Docker, Docker Compose and Git
apt-get install -y docker.io docker-compose-v2 git

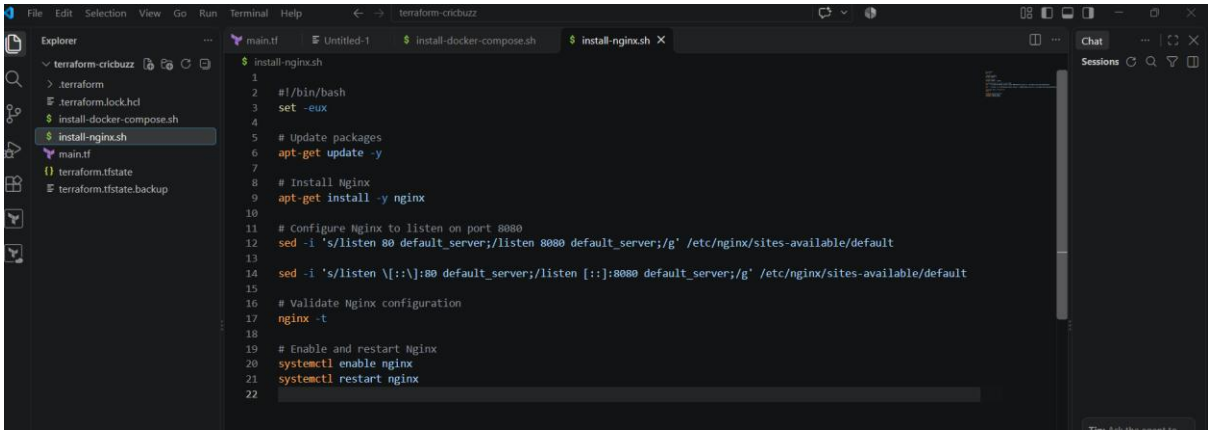
# Enable and start Docker
systemctl enable docker
systemctl start docker

# Clone repository if it does not exist
mkdir -p /opt/cricbuzz
cd /opt/cricbuzz

if [ ! -d cricbuzz-docker-hello-world ]; then
    git clone https://github.com/betawins/cricbuzz-docker-hello-world.git
fi

cd /opt/cricbuzz/cricbuzz-docker-hello-world

# Create Docker Compose file
cat > docker-compose.yml <<'EOF'
services:
  cricbuzz-app:
    image: ehcr.io/infrastructure-as-code/hello-world:latest
```



```
#!/bin/bash
set -eux

# Update packages
apt-get update -y

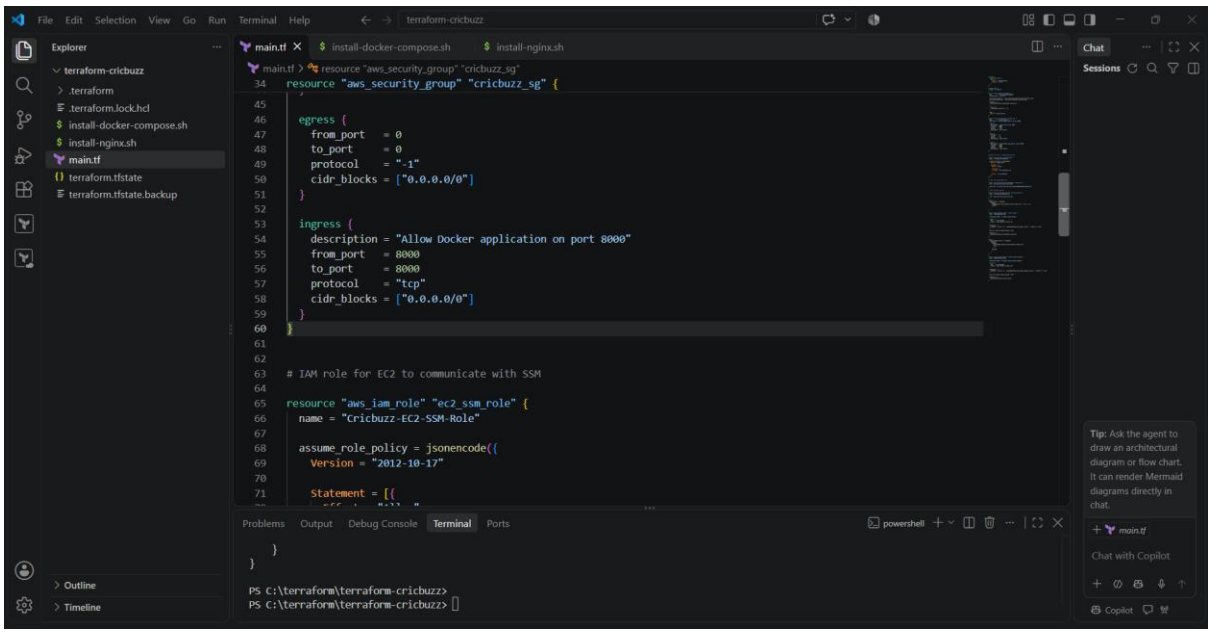
# Install Nginx
apt-get install -y nginx

# Configure Nginx to listen on port 8080
sed -i 's/listen 80 default_server;/listen 8080 default_server;/g' /etc/nginx/sites-available/default

sed -i 's/listen \[::\]:80 default_server;/listen [::]:8080 default_server;/g' /etc/nginx/sites-available/default

# Validate Nginx configuration
nginx -t

# Enable and restart Nginx
systemctl enable nginx
systemctl restart nginx
```



```
resource "aws_security_group" "cricbuzz_sg" {

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow Docker application on port 8000"
    from_port   = 8000
    to_port     = 8000
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}


# IAM role for EC2 to communicate with SSM

resource "aws_iam_role" "ec2_ssm_role" {
  name = "Cricbuzz-EC2-SSM-Role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"

    Statement = [{
```

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

PS C:\terraform\terraform-cricbuzz> terraform apply
aws_iam_role.ec2_ssm_role: Refreshing state... [id=Cricbuzz-EC2-SSM-Role]
aws_security_group.cricbuzz_sg: Refreshing state... [id=sg-06b32ce35112c41a6]
aws_iam_role_policy_attachment.ssm_policy: Refreshing state... [id=Cricbuzz-EC2-SSM-Role-20261102074223319300000001]
aws_iam_instance_profile.ec2_ssm_profile: Refreshing state... [id=Cricbuzz-EC2-SSM-Profile]
aws_instance.cricbuzz: Refreshing state... [id=i-02e7538bb06c03d92]
aws_ssm_association.install_nginx: Refreshing state... [id=c674103c-af87-4bee-a1f1-913608a256f2]
aws_ssm_association.install_docker_compose: Refreshing state... [id=82b58d30-2328-48d7-885c-d0ebe8d38fe4]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:

  # aws_instance.cricbuzz will be updated in-place
  ~ resource "aws_instance" "cricbuzz" {
        id                                   = "i-02e7538bb06c03d92"
        tags                                 = {
            "Name" = "Cricbuzz-Server"
        }
      ~ user_data                            = "f17cb963d27071933ffa416222a2f83566b09305" -> "3b899bd6407bd4f7107f4caa9b769e2cf576232f"
        # (39 unchanged attributes hidden)

        # (8 unchanged blocks hidden)
    }

  # aws_security_group.cricbuzz_sg will be updated in-place
```

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

        # (1 unchanged block hidden)
    }

Plan: 0 to add, 3 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.cricbuzz_sg: Modifying... [id=sg-06b32ce35112c41a6]
aws_security_group.cricbuzz_sg: Modifications complete after 2s [id=sg-06b32ce35112c41a6]
aws_instance.cricbuzz: Modifying... [id=i-02e7538bb06c03d92]
aws_instance.cricbuzz: Modifications complete after 3s [id=i-02e7538bb06c03d92]
aws_ssm_association.install_docker_compose: Modifying... [id=82b58d30-2328-48d7-885c-d0ebe8d38fe4]
aws_ssm_association.install_docker_compose: Modifications complete after 2s [id=82b58d30-2328-48d7-885c-d0ebe8d38fe4]

Apply complete! Resources: 0 added, 3 changed, 0 destroyed.
```

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

PS C:\terraform\terraform-cricbuzz> $commandId = aws ssm send-command `
>>   --instance-ids i-02e7538bb06c03d92 `
>>   --document-name "AWS-RunShellScript" `
>>   --parameters 'commands=["docker ps","docker compose -f /opt/cricbuzz/cricbuzz-docker-hello-world/docker-compose.yml ps"]' `
>>   --query "Command.CommandId" `
>>   --output text
>>
>> Start-Sleep -Seconds 10
>>
>> aws ssm get-command-invocation `
>>   --command-id $commandId `
>>   --instance-id i-02e7538bb06c03d92 `
>>   --output json
{
    "CommandId": "305f311d-b7c4-466a-b9b3-84958eadb9a8",
    "InstanceId": "i-02e7538bb06c03d92",
    "Comment": "",
    "DocumentName": "AWS-RunShellScript",
    "DocumentVersion": "$DEFAULT",
    "PluginName": "aws:runShellScript",
    "ResponseCode": 0,
    "ExecutionStartDateTime": "2026-11-02T10:31:35.402Z",
    "ExecutionElapsedTime": "PT0.14S",
    "ExecutionEndDateTime": "2026-11-02T10:31:35.402Z",
    "Status": "Success",
    "StatusDetails": "Success",
    "StandardOutputContent": "CONTAINER ID   IMAGE                                                    COMMAND          CREATED          STATUS
```



4. **Make the web server point to the above service so that 8080 returns the response from the Docker service.**

Step 1: Understand the requirement

At this stage:

- Nginx was listening on port 8080.
- The Docker application was running on port 8000.

The requirement was to forward requests received on port 8080 to the Docker application on port 8000.

Step 2: Create the reverse proxy script

Created a file named:

configure-reverse-proxy.sh

Used the following script:

```
#!/bin/bash

set -eux


cat > /etc/nginx/sites-available/default <<'EOF'

server {

  listen 8080 default_server;

  listen [::]:8080 default_server;


  server_name _;
```

```
    location / {
        proxy_pass http://127.0.0.1:8000;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF

nginx -t

systemctl restart nginx
systemctl is-active nginx
```

Explanation:

- Configured Nginx to listen on port 8080.
- Configured proxy_pass to forward requests to 127.0.0.1:8000.
- Forwarded the original request headers.
- Validated the Nginx configuration.
- Restarted Nginx.

Step 3: Configure the Terraform local variable

Added the following configuration in main.tf:

```
locals {
  reverse_proxy_script = trimspace(
    replace(
      replace(file("${path.module}/configure-reverse-proxy.sh"), "\r\n", "\n"),
      "\uFEFF",
```

```
    """
  )
 )
}
```

Step 4: Execute using Terraform SSM Association

Created the resource:

aws_ssm_association.configure_reverse_proxy

Used:

- Document: AWS-RunShellScript
- Association name: cricbuzz-configure-reverse-proxy
- Target: aws_instance.cricbuzz.id
- Timeout: 600 seconds

Executed the script using:

printf '%s' '${base64encode(local.reverse_proxy_script)}' | base64 -d | bash

Step 5: Verify the reverse proxy

Executed:

sudo nginx -t

sudo systemctl is-active nginx

Step 6: Browser verification

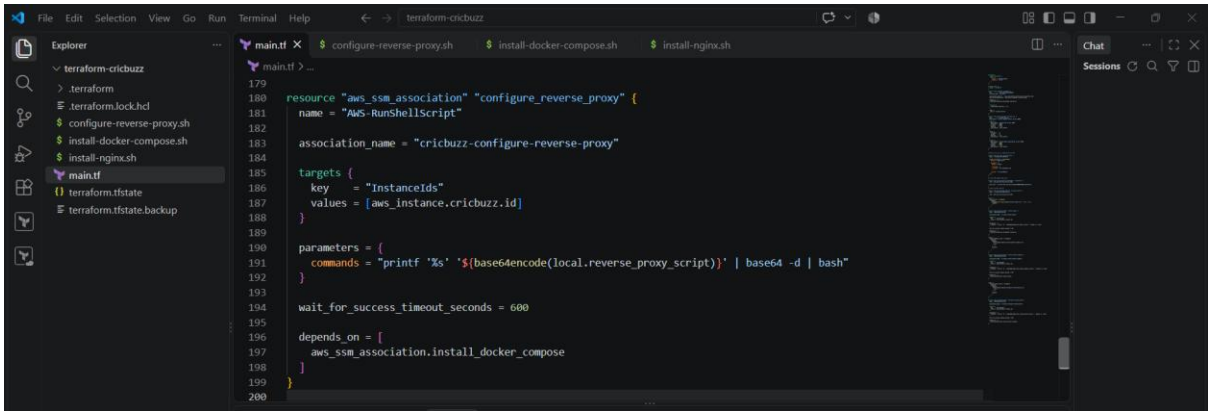
Opened:

http://100.26.48.41:8080

Nginx forwarded the request to the Docker application running on port 8000.

The Hello World application was displayed through port 8080.

Result: Nginx reverse proxy configured successfully. Requests received on port 8080 were forwarded to the Docker application on port 8000.



```
resource "aws_ssm_association" "configure_reverse_proxy" {
  name = "AWS-RunShellScript"

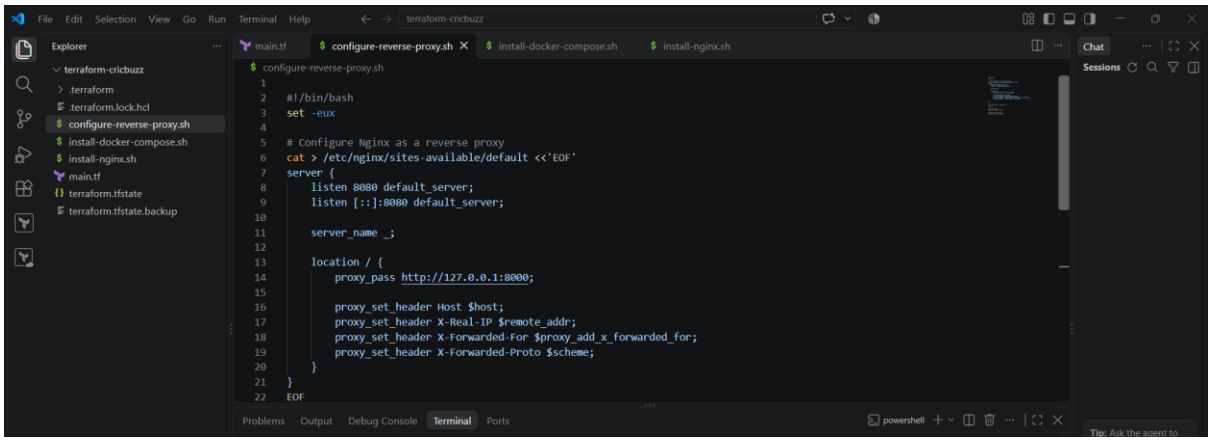
  association_name = "cricbuzz-configure-reverse-proxy"

  targets {
    key    = "InstanceIds"
    values = [aws_instance.cricbuzz.id]
  }

  parameters = {
    commands = "printf '%s' '${base64encode(local.reverse_proxy_script)}' | base64 -d | bash"
  }

  wait_for_success_timeout_seconds = 600

  depends_on = [
    aws_ssm_association.install_docker_compose
  ]
}
```



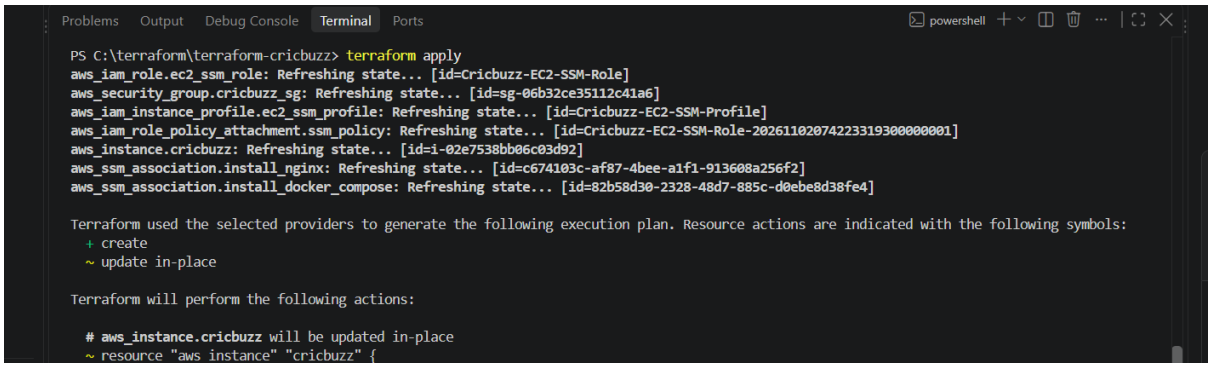
```
#!/bin/bash
set -eux

# Configure Nginx as a reverse proxy
cat > /etc/nginx/sites-available/default <<'EOF'
server {
    listen 8080 default_server;
    listen [::]:8080 default_server;

    server_name _;

    location / {
        proxy_pass http://127.0.0.1:8000;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
```

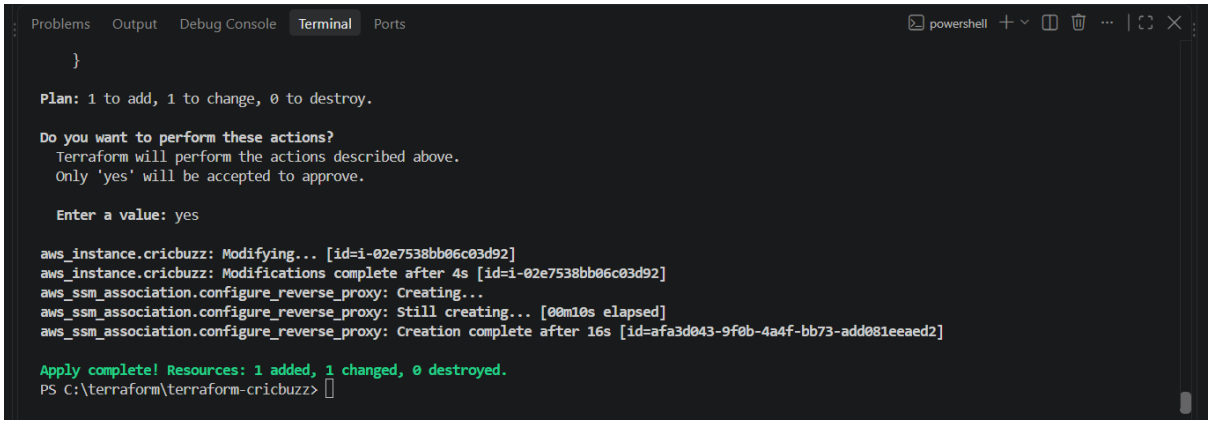


```
PS C:\terraform\terraform-cricbuzz> terraform apply
aws_iam_role.ec2_ssm_role: Refreshing state... [id=Cricbuzz-EC2-SSM-Role]
aws_security_group.cricbuzz_sg: Refreshing state... [id=sg-06b32ce35112c41a6]
aws_iam_instance_profile.ec2_ssm_profile: Refreshing state... [id=Cricbuzz-EC2-SSM-Profile]
aws_iam_role_policy_attachment.ssm_policy: Refreshing state... [id=Cricbuzz-EC2-SSM-Role-20261102074223319300000001]
aws_instance.cricbuzz: Refreshing state... [id=i-02e7538bb06c03d92]
aws_ssm_association.install_nginx: Refreshing state... [id=c674103c-af87-4bee-a1f1-913608a256f2]
aws_ssm_association.install_docker_compose: Refreshing state... [id=82b58d30-2328-48d7-885c-d0ebe8d38fe4]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create
  ~ update in-place

Terraform will perform the following actions:

  # aws_instance.cricbuzz will be updated in-place
  ~ resource "aws_instance" "cricbuzz" {
```



```
    }

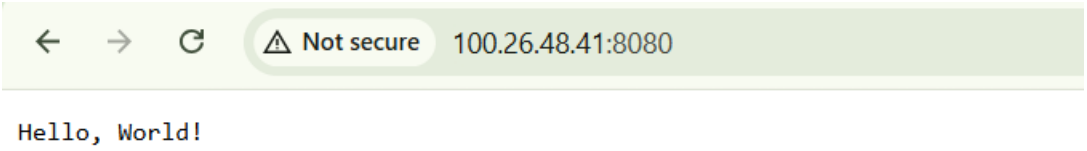
Plan: 1 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.cricbuzz: Modifying... [id=i-02e7538bb06c03d92]
aws_instance.cricbuzz: Modifications complete after 4s [id=i-02e7538bb06c03d92]
aws_ssm_association.configure_reverse_proxy: Creating...
aws_ssm_association.configure_reverse_proxy: Still creating... [00m10s elapsed]
aws_ssm_association.configure_reverse_proxy: Creation complete after 16s [id=afa3d043-9f0b-4a4f-bb73-add081eeaed2]

Apply complete! Resources: 1 added, 1 changed, 0 destroyed.
PS C:\terraform\terraform-cricbuzz>
```



5. Configure to Save the Logs in the location /var/log/cb.log

Step 1: Understand the requirement

The requirement was to store both Nginx access logs and error logs in a custom log file:

/var/log/cb.log

Step 2: Update the Nginx configuration

Updated the Nginx configuration file:

/etc/nginx/sites-available/default

The final server configuration included:

```
server {
    listen 8080 default_server;
    listen [::]:8080 default_server;

    server_name _;

    access_log /var/log/cb.log;
    error_log /var/log/cb.log;

    location / {
        proxy_pass http://127.0.0.1:8000;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

Important: We updated the existing reverse-proxy configuration instead of creating a separate Nginx server configuration.

Step 3: Execute the updated configuration through SSM

Updated the existing Terraform association:

aws_ssm_association.configure_reverse_proxy

Used the same SSM execution method to apply the updated configuration.

Step 4: Verify the configuration

Executed:

sudo nginx -t

sudo systemctl restart nginx

sudo systemctl is-active nginx

Step 5: Generate application traffic

Opened:

http://100.26.48.41:8080

This generated HTTP requests that were recorded in the Nginx log file.

Step 6: Verify the log file

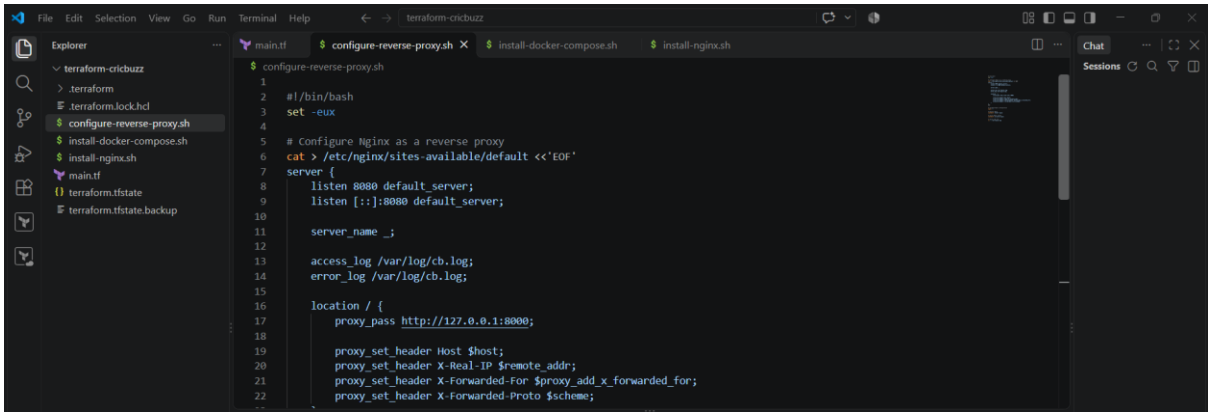
Executed:

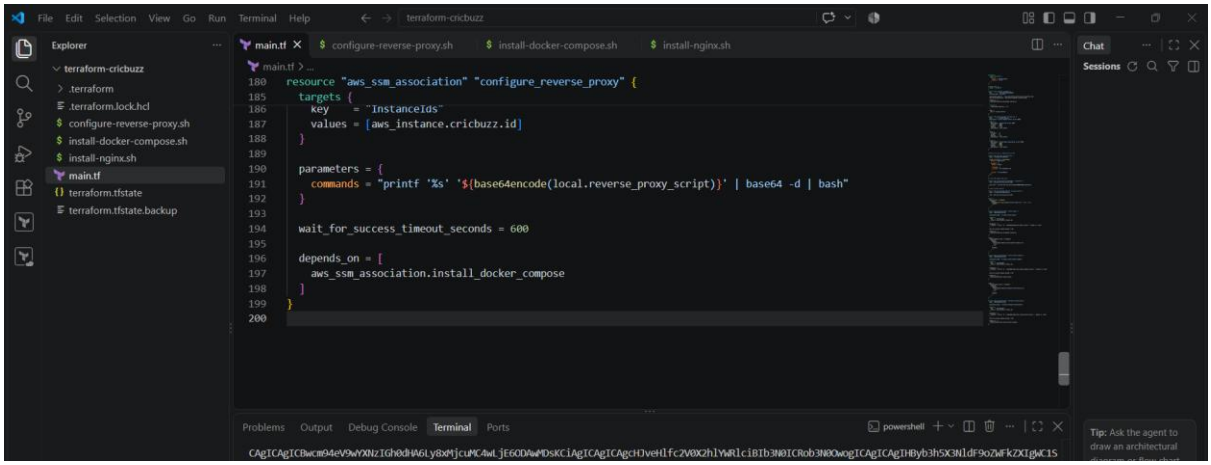
sudo ls -lh /var/log/cb.log

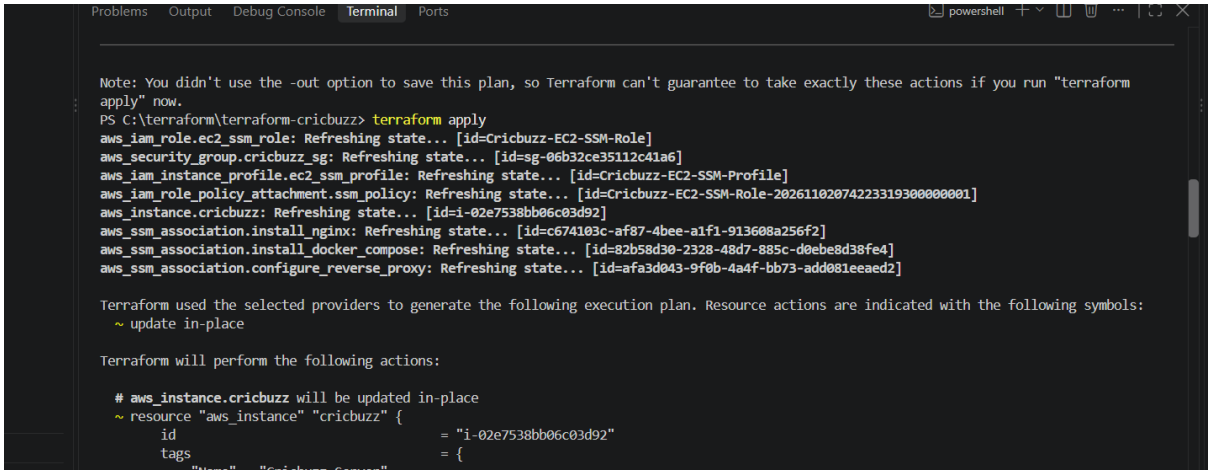
sudo tail -10 /var/log/cb.log

Verified that the file existed and contained application request entries, including a successful GET / request with HTTP status 200.

Result: Nginx access and error logs successfully configured to use /var/log/cb.log.


File Edit Selection View Go Run Terminal Help
terraform-cricbuzz
Explorer
terraform-cricbuzz
.terraform
.terraform.lock.hcl
configure-reverse-proxy.sh
install-docker-compose.sh
install-nginx.sh
main.tf
terraform.tfstate
terraform.tfstate.backup
main.tf
configure-reverse-proxy.sh
install-docker-compose.sh
install-nginx.sh
Chat
Sessions
#!/bin/bash
set -eux
Configure Nginx as a reverse proxy
cat > /etc/nginx/sites-available/default <<'EOF'
server {
listen 8080 default_server;
listen [::]:8080 default_server;
server_name _;
access_log /var/log/cb.log;
error_log /var/log/cb.log;
location / {
proxy_pass http://127.0.0.1:8000;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;


File Edit Selection View Go Run Terminal Help
terraform-cricbuzz
Explorer
terraform-cricbuzz
.terraform
.terraform.lock.hcl
configure-reverse-proxy.sh
install-docker-compose.sh
install-nginx.sh
main.tf
terraform.tfstate
terraform.tfstate.backup
main.tf
configure-reverse-proxy.sh
install-docker-compose.sh
install-nginx.sh
Chat
Sessions
resource "aws_ssm_association" "configure_reverse_proxy" {
targets {
key = "InstanceIds"
values = [aws_instance.cricbuzz.id]
}
parameters = {
commands = "printf '%s' '${base64encode(local.reverse_proxy_script)}' | base64 -d | bash"
}
wait_for_success_timeout_seconds = 600
depends_on = [
aws_ssm_association.install_docker_compose
]
}
Problems Output Debug Console Terminal Ports
powershell
Tip: Ask the agent to draw an architectural


Problems Output Debug Console Terminal Ports
powershell
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\terraform\terraform-cricbuzz> terraform apply
aws_iam_role.ec2_ssm_role: Refreshing state... [id=Cricbuzz-EC2-SSM-Role]
aws_security_group.cricbuzz_sg: Refreshing state... [id=sg-06b32ce35112c41a6]
aws_iam_instance_profile.ec2_ssm_profile: Refreshing state... [id=Cricbuzz-EC2-SSM-Profile]
aws_iam_role_policy_attachment.ssm_policy: Refreshing state... [id=Cricbuzz-EC2-SSM-Role-20261102074223319300000001]
aws_instance.cricbuzz: Refreshing state... [id=i-02e7538bb06c03d92]
aws_ssm_association.install_nginx: Refreshing state... [id=c674103c-af87-4bee-a1f1-913608a256f2]
aws_ssm_association.install_docker_compose: Refreshing state... [id=82b58d30-2328-48d7-885c-d0ebe8d38fe4]
aws_ssm_association.configure_reverse_proxy: Refreshing state... [id=afa3d043-9f0b-4a4f-bb73-add081eeaed2]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
~ update in-place
Terraform will perform the following actions:
aws_instance.cricbuzz will be updated in-place
~ resource "aws_instance" "cricbuzz" {
id = "i-02e7538bb06c03d92"
tags = {

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

N0ZW1jdGwgcmVzdGFydCBuZ2lueAoKIyBWZXJpZnkgTmdpbnggc3RhdHVzCnN5c3RlbWN0bCBpcy1hY3RpdmUgbmdpbngKCiMgVmVyaWZ5IGxvZyBmaWxlCmxzIC1sIC92YXIvbG9nL2NiL2NiLmxvZw==' | base64 -d | bash"
        }
        tags                            = {}
        # (14 unchanged attributes hidden)

        # (1 unchanged block hidden)
    }

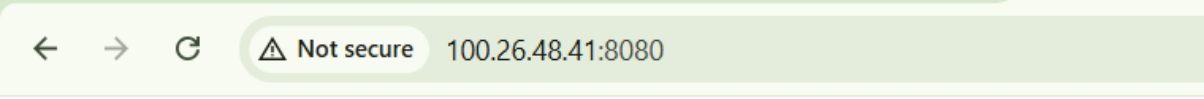
Plan: 0 to add, 2 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.cricbuzz: Modifying... [id=i-02e7538bb06c03d92]
aws_instance.cricbuzz: Modifications complete after 4s [id=i-02e7538bb06c03d92]
aws_ssm_association.configure_reverse_proxy: Modifying... [id=afa3d043-9f0b-4a4f-bb73-add081eeaed2]
aws_ssm_association.configure_reverse_proxy: Modifications complete after 2s [id=afa3d043-9f0b-4a4f-bb73-add081eeaed2]

Apply complete! Resources: 0 added, 2 changed, 0 destroyed.
```



Hello, World!

```
Problems  Output  Debug Console  Terminal  Ports                                                    powershell

PS C:\terraform\terraform-cricbuzz> $commandId = aws ssm send-command `
>>   --instance-ids i-02e7538bb06c03d92 `
>>   --document-name "AWS-RunShellScript" `
>>   --parameters 'commands=["ls -lh /var/log/cb.log","tail -10 /var/log/cb.log"]' `
>>   --query "Command.CommandId" `
>>   --output text
>>
>> Start-Sleep -Seconds 5
>>
>> aws ssm get-command-invocation `
>>   --command-id $commandId `
>>   --instance-id i-02e7538bb06c03d92 `
>>   --output json
{
    "CommandId": "58d8c7a7-afd7-46e7-98d6-5aa5b305137e",
    "InstanceId": "i-02e7538bb06c03d92",
    "Comment": "",
    "DocumentName": "AWS-RunShellScript",
    "DocumentVersion": "$DEFAULT",
    "PluginName": "aws:runShellScript",
    "ResponseCode": 0,
    "ExecutionStartDateTime": "2026-11-02T10:42:16.472Z",
    "ExecutionElapsedTime": "PT0.016S",
    "ExecutionEndDateTime": "2026-11-02T10:42:16.472Z",
    "Status": "Success",
    "StatusDetails": "Success",
    "StandardOutputContent": "-rw-r--r-- 1 root root 191 Nov  2 10:42 /var/log/cb.log\n103.203.172.230 - - [02/Nov/2026:10:42:06 +0000] \"GET /
HTTP/1.1\" 200 13 \"-\" \"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/156.0.0.0 Safari/537.36\"\n"
,
    "StandardOutputUrl": "",
    "StandardErrorContent": "",
    "StandardErrorUrl": "",
    "CloudWatchOutputConfig": {
        "CloudWatchLogGroupName": "",
        "CloudWatchOutputEnabled": false
```

6. Write the Log rotate configuration for the above file

Step 1: Understand the requirement

Log files continuously grow as application requests are received.

Logrotate helps manage log files by rotating, compressing and retaining older log files.

The requirement was to configure daily rotation for:

/var/log/cb.log

Step 2: Create the Logrotate script

Created a file named:

configure-logrotate.sh

Used the following final corrected script:

```
#!/bin/bash
set -eux

cat > /etc/logrotate.d/cb <<'EOF'
/var/log/cb.log {
   daily
   rotate 7
   missingok
   notifempty
   compress
   delaycompress
   su root root
   create 0644 root root
   sharedscripts
   postrotate
      systemctl reload nginx
   endscript
}
EOF
```

logrotate -d /etc/logrotate.d/cb

Step 3: Understand the Logrotate configuration

Directive	Purpose
daily	Rotate logs daily
rotate 7	Retain seven rotated log files
missingok	Ignore missing log files
notifempty	Do not rotate empty files
compress	Compress rotated files
delaycompress	Delay compression until the next rotation
su root root	Run rotation with root user and group
create 0644 root root	Create a new log file with specified permissions and ownership
sharedscripts	Execute the post-rotation script once
postrotate	Execute a command after rotation
systemctl reload nginx	Reload Nginx after rotation

Step 4: Configure the Terraform local variable

Added the following local variable in main.tf:

```
locals {
 logrotate_script = trimspace(
  replace(
   replace(file("${path.module}/configure-logrotate.sh"), "\r\n", "\n"),
   "\uFEFF",
   ""
  )
 )
}
```

Step 5: Execute using Terraform SSM Association

Created the Terraform resource:

aws_ssm_association.configure_logrotate

Configured it with:

- Document: AWS-RunShellScript
- Association name: cricbuzz-configure-logrotate
- Target: aws_instance.cricbuzz.id
- Timeout: 600 seconds

The execution command was:

printf '%s' '${base64encode(local.logrotate_script)}' | base64 -d | bash

The association was configured to depend on:

aws_ssm_association.configure_reverse_proxy

Step 6: Resolve the Logrotate permission issue

During validation, Logrotate reported a permission issue related to the parent directory.

To resolve this, added:

su root root

to /etc/logrotate.d/cb.

Updated the Terraform script and applied the configuration again.

This resolved the permission issue.

Step 7: Verify Logrotate configuration

Executed:

sudo logrotate -d /etc/logrotate.d/cb

The debug output confirmed that the configuration was processed without the previous permission error.

Also verified the log file:

sudo ls -l /var/log/cb.log

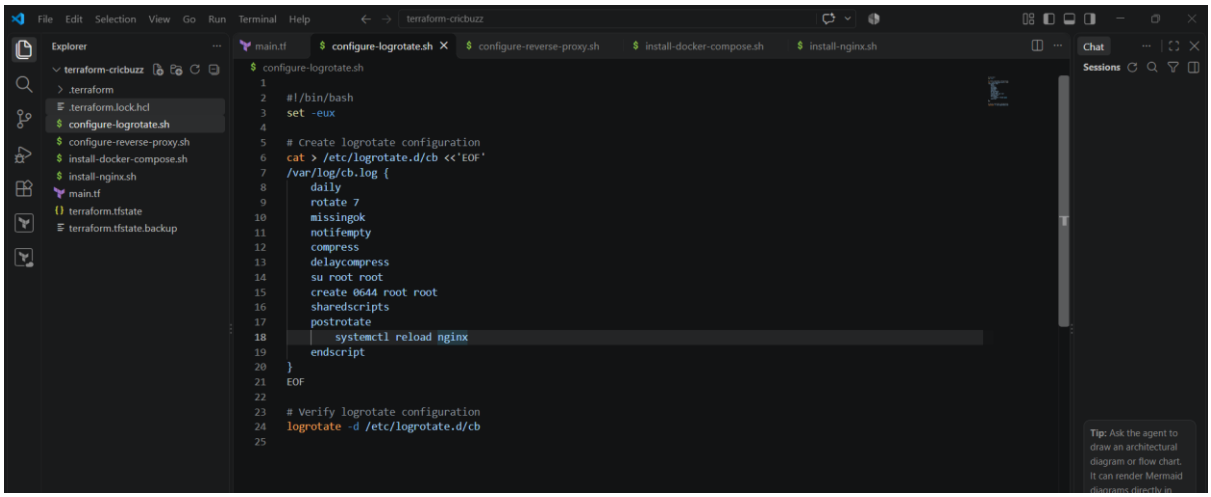
The SSM association completed successfully.

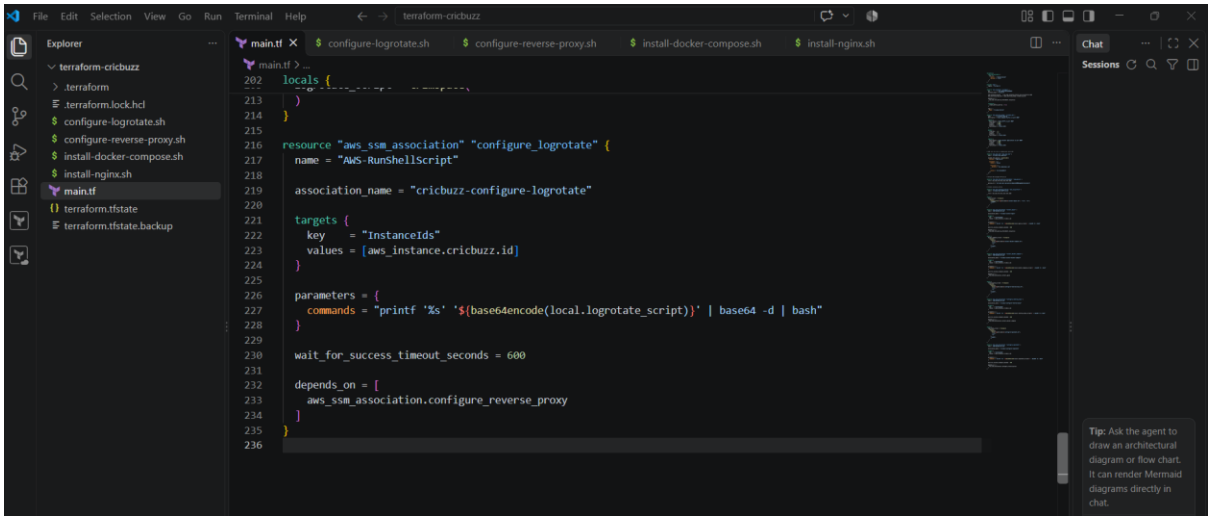
Final Association ID: d4e48efa-9750-49a9-bc5e-a2e41a0facf7

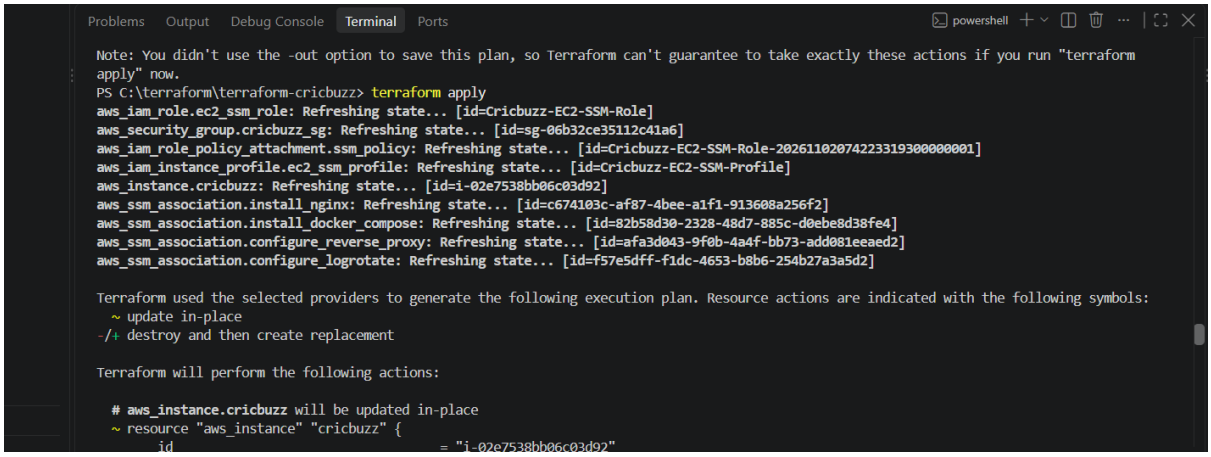
SSM Execution Result: Response Code 0 – Status Success.

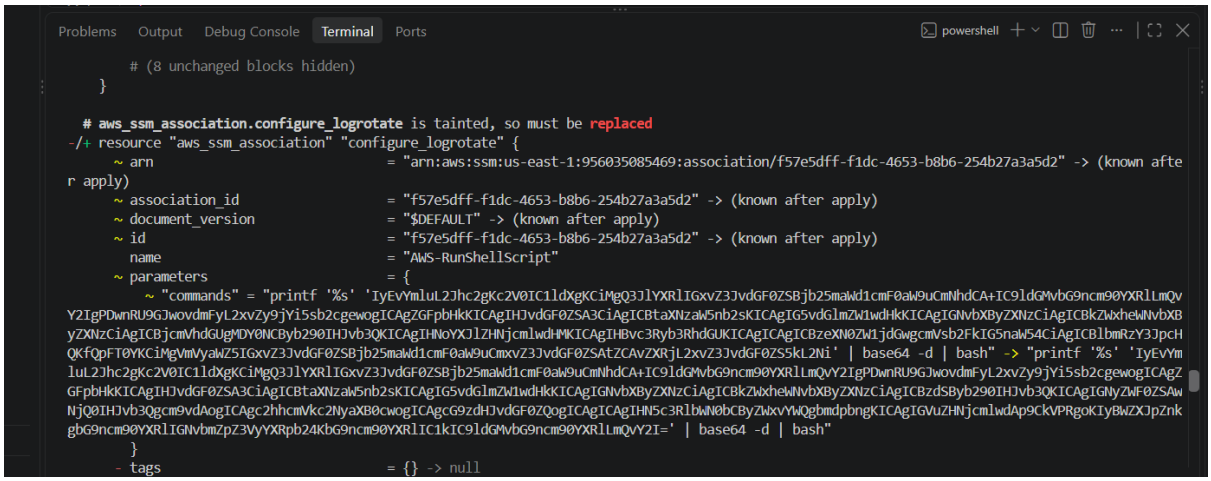
Note: The -d option runs Logrotate in debug mode. It validates the configuration but does not perform an actual rotation.

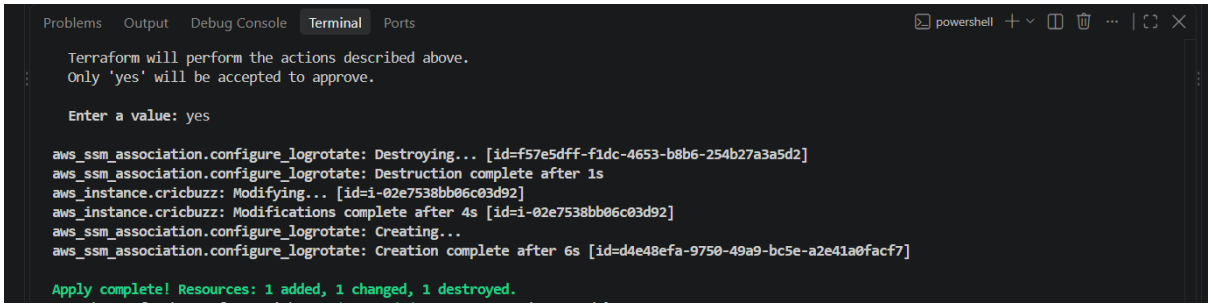
Result: Logrotate configured successfully for /var/log/cb.log, with daily rotation and seven retained rotated files.






Problems Output Debug Console Terminal Ports
powershell
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
PS C:\terraform\terraform-cricbuzz> terraform apply
aws_iam_role.ec2_ssm_role: Refreshing state... [id=Cricbuzz-EC2-SSM-Role]
aws_security_group.cricbuzz_sg: Refreshing state... [id=sg-06b32ce35112c41a6]
aws_iam_role_policy_attachment.ssm_policy: Refreshing state... [id=Cricbuzz-EC2-SSM-Role-20261102074223319300000001]
aws_iam_instance_profile.ec2_ssm_profile: Refreshing state... [id=Cricbuzz-EC2-SSM-Profile]
aws_instance.cricbuzz: Refreshing state... [id=i-02e7538bb06c03d92]
aws_ssm_association.install_nginx: Refreshing state... [id=c674103c-af87-4bee-a1f1-913608a256f2]
aws_ssm_association.install_docker_compose: Refreshing state... [id=82b58d30-2328-48d7-885c-d0ebe8d38fe4]
aws_ssm_association.configure_reverse_proxy: Refreshing state... [id=afa3d043-9f0b-4a4f-bb73-add081eeaed2]
aws_ssm_association.configure_logrotate: Refreshing state... [id=f57e5dff-f1dc-4653-b8b6-254b27a3a5d2]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
~ update in-place
-/+ destroy and then create replacement
Terraform will perform the following actions:
aws_instance.cricbuzz will be updated in-place
~ resource "aws_instance" "cricbuzz" {
id = "i-02e7538bb06c03d92"


Problems Output Debug Console Terminal Ports
powershell
(8 unchanged blocks hidden)
}
aws_ssm_association.configure_logrotate is tainted, so must be replaced
-/+ resource "aws_ssm_association" "configure_logrotate" {
~ arn = "arn:aws:ssm:us-east-1:956035085469:association/f57e5dff-f1dc-4653-b8b6-254b27a3a5d2" -> (known after apply)
~ association_id = "f57e5dff-f1dc-4653-b8b6-254b27a3a5d2" -> (known after apply)
~ document_version = "$DEFAULT" -> (known after apply)
~ id = "f57e5dff-f1dc-4653-b8b6-254b27a3a5d2" -> (known after apply)
name = "AWS-RunShellScript"
~ parameters = {
~ "commands" = "printf '%s' 'IyEvYmluL2Jhc2gKc2V0IC1ldXgKCiMgQ3JlYXRlIGxvZ3JvdGF0ZSBjb25maWd1cmF0aW9uCmNhdCA+IC9ldGMvbG9ncm90YXRllmQvY2IgPDwnRU9GJwovdmFyL2xvZy9jYi5sb2cgewogICAgZGFpbHkKICAgIHJvdGF0ZSA3CiAgICBtaXNzaW5nb2sKICAgIG5vdGlmeXNzCiAgICBkcHJvdGVjdGVkCiAgICBzaGFyZWRzY3JpcHQKICAgIHBvc3Jvdgo9dmFyL2xvZy9jYi5sb2cgewogICAgZGFpbHkKICAgIHJvdGF0ZSA3CiAgICBtaXNzaW5nb2sKICAgIG5vdGlmeXNzCiAgICBzdSByb290IHJvb3QKICAgIGNyZWF0ZSAwNjQ0IHJvb3Qgcm9vdAogICAgc2hhcmVkc2NyaXB0cwogICAgcG9zdHJvdGF0ZQogICAgIHN5c3RlbWN0bCByZWxvYWQgbmdpbngKICAgIGVuZHNjcmlwdAp9CkVPRgoKIyBWZXJpZnkgbG9ncm90YXRlIGNvbmZpZ3VyYXRpb24KbG9ncm90YXRlIC1kIC9ldGMvbG9ncm90YXRlLmQvY2I=' | base64 -d | bash" -> "printf '%s' 'IyEvYmluL2Jhc2gKc2V0IC1ldXgKCiMgQ3JlYXRlIGxvZ3JvdGF0ZSBjb25maWd1cmF0aW9uCmNhdCA+IC9ldGMvbG9ncm90YXRllmQvY2IgPDwnRU9GJwovdmFyL2xvZy9jYi5sb2cgewogICAgZGFpbHkKICAgIHJvdGF0ZSA3CiAgICBtaXNzaW5nb2sKICAgIG5vdGlmeXNzCiAgICBzdSByb290IHJvb3QKICAgIGNyZWF0ZSAwNjQwIHJvb3Qgcm9vdAogICAgc2hhcmVkc2NyaXB0cwogICAgcG9zdHJvdGF0ZQogICAgIHN5c3RlbWN0bCByZWxvYWQgbmdpbngKICAgIGVuZHNjcmlwdAp9CkVPRgoKIyBWZXJpZnkgbG9ncm90YXRlIGNvbmZpZ3VyYXRpb24KbG9ncm90YXRlIC1kIC9ldGMvbG9ncm90YXRlLmQvY2I="
}
- tags = {} -> null


Problems Output Debug Console Terminal Ports
powershell
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
aws_ssm_association.configure_logrotate: Destroying... [id=f57e5dff-f1dc-4653-b8b6-254b27a3a5d2]
aws_ssm_association.configure_logrotate: Destruction complete after 1s
aws_instance.cricbuzz: Modifying... [id=i-02e7538bb06c03d92]
aws_instance.cricbuzz: Modifications complete after 4s [id=i-02e7538bb06c03d92]
aws_ssm_association.configure_logrotate: Creating...
aws_ssm_association.configure_logrotate: Creation complete after 6s [id=d4e48efa-9750-49a9-bc5e-a2e41a0facf7]
Apply complete! Resources: 1 added, 1 changed, 1 destroyed.

```
PS C:\terraform\terraform-cricbuzz> $commandId = aws ssm send-command `
>>   --instance-ids i-02e7538bb06c03d92 `
>>   --document-name "AWS-RunShellScript" `
>>   --parameters 'commands=["cat /etc/logrotate.d/cb","logrotate -d /etc/logrotate.d/cb 2>&1","ls -lh /var/log/cb.log"]' `
>>   --query "Command.CommandId" `
>>   --output text
>>
>> Start-Sleep -Seconds 5
>>
>> aws ssm get-command-invocation `
>>   --command-id $commandId `
>>   --instance-id i-02e7538bb06c03d92 `
>>   --output json
{
    "CommandId": "eacee765-2c1a-4d7e-8a56-826436942fb8",
    "InstanceId": "i-02e7538bb06c03d92",
    "Comment": "",
    "DocumentName": "AWS-RunShellScript",
    "DocumentVersion": "$DEFAULT",
    "PluginName": "aws:runShellScript",
    "ResponseCode": 0,
    "ExecutionStartDateTime": "2026-11-02T11:11:27.142Z",
    "ExecutionElapsedTime": "PT0.014S",
    "ExecutionEndDateTime": "2026-11-02T11:11:27.142Z",
    "Status": "Success",
    "StatusDetails": "Success",
    "StandardOutputContent": "/var/log/cb.log {\n    daily\n    rotate 7\n    missingok\n    notifempty\n    compress\n    delaycompress\n    s
u root root\n    create 0644 root root\n    sharedscripts\n    postrotate\n        systemctl reload nginx\n    endscript\n}\nwarning: logrotate
 in debug mode does nothing except printing debug messages!  Consider using verbose mode (-v) instead if this is not what you want.\n\nreading
config file /etc/logrotate.d/cb\nReading state from file: /var/lib/logrotate/status\nstate file /var/lib/logrotate/status does not exist\nAlloc
ating hash table for state file, size 64 entries\n\nHandling 1 logs\n\nrotating pattern: /var/log/cb.log after 1 days empty log files are not r
```

